

Web-Technologien

HTML-Dokumente

- Syntax und Struktur
- URLs und Hyperlinks
- Semantische Container
- Formulare

Letzte Aktualisierung: 23. Februar 2025



Lernziele

- Wissen wie der syntaktische Aufbau von HTML ist.
- Verstehen wie HTML Informationen strukturiert und wie der Aufbau von HTML-Dokumenten ist.
- Wissen wie URLs in HTML-Dokumenten verwendet werden.
- Die vorgestellten HTML-Elemente kennen und anwenden können.

HTML-Dokument: Erstes Beispiel

```
<html>
  <head>
    <title>Hello World</title>
  </head>
  <body>
    <p>Hallo WorldWideWeb!</p>
  </body>
</html>
```

- **Strukturierter** Aufbau durch Tags.
- Einzurücken der Codes zur besseren Lesbarkeit.

Wer schaut zu HTML?

- Das WHATWG-Konsortium (früher W3C) ist für die Standardisierung zuständig <https://whatwg.org>.
- HTML Living Standard: <https://html.spec.whatwg.org/multipage/>.
- Verschiedene Themen rund um HTML5 werden in eigenen Spezifikationen verabschiedet bzw. sind in der Entwicklung (AAM, HTML Extension Specifications, ARIA, ...). <https://www.w3.org/standards/techs/html#stds>

Historisches: am Anfang war SGML

- Entstanden Ende der 80er Jahre im CERN.
- Wurzeln bei SGML (**S**tructure **G**eneralized **M**arkup **L**anguage) aus den 60ern.
- HTML ist eine "Auszeichnungssprache" (**H**yper**T**ext **M**arkup **L**anguage)
=> deklaratives vs. imperatives Programmieren.
- Idee: "Inhalt/Information strukturieren".
- Strukturierung mittels besonderer Auszeichnungen, den sogenannten Markups.
 - Strukturen sind z.B. Titel, Überschrift, Absatz, Block, etc.

Elemente der Dokumentenstruktur

- Verschachtelung der Elemente ineinander (Baum).
- Elementanfang und -ende wird durch öffnenden `<tag>` und schliessenden `</tag>` gekennzeichnet.
- Äusserstes Element immer `<html>...</html>`
- Verschachtelung **muss** korrekt sein.

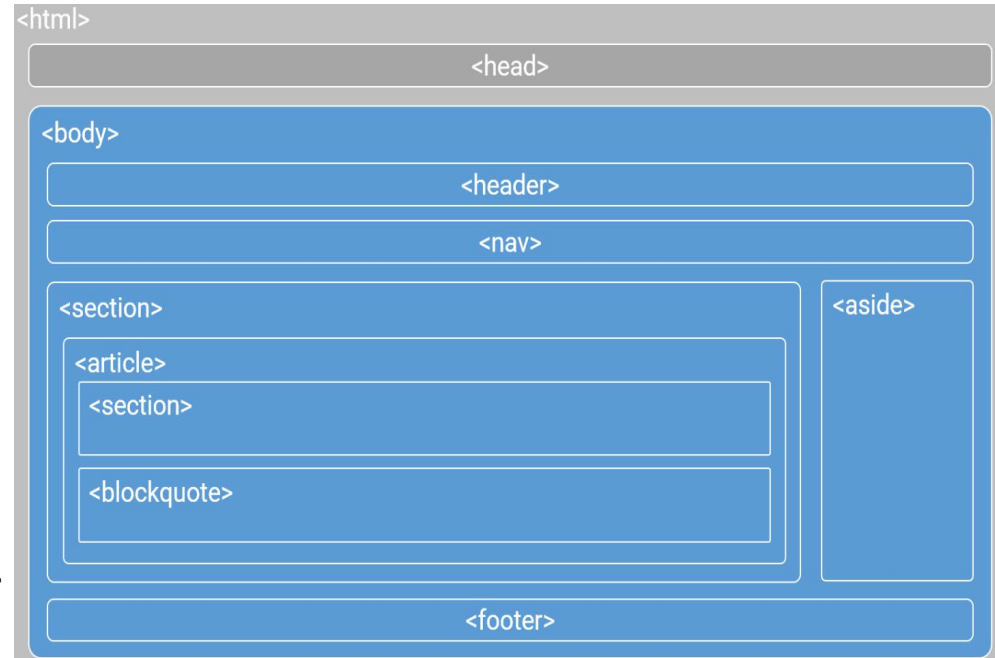


Bild: <https://www.seonative.de/seo-ueberschriften-und-html5-sections/>

HTML-Markup

- Öffnende und zugehörige schliessende Tags:

```
<body> ... </body>
```

- Alleinstehende Tags:

```
<br> oder <br/>
```

- Tag-Attribute:
(immer in Anführungszeichen) .

```
<a href="https://www.hslu.ch">HSLU</a>
```

Grundlegende Dokumentenstruktur in HTML

HTML-Dokumente besitzen in der Regel folgende Dokumentenstruktur:

```
<html>
  <head>
    <title>Hello World</title>
  </head>
  <body>
    <p>Hallo WorldWideWeb!</p>
  </body>
</html>
```

- Erweiterungen (**Attribute**), beispielsweise Stilangaben zur Formatierung etc., können zusätzlich angegeben werden.

<head>-Element

- Enthält Informationen wie:
 - Metainformation (<meta>).
 - Titel (<title>).
 - Stil (<style>).
 - Script-Definitionen (<script>).
 - etc.
- Diese Informationen werden vom Webbrowser nicht im Dokumentenbereich angezeigt.

<body>-Element

- HTML-Element <body> kennzeichnet den Anfang und das Ende des sichtbaren Inhalts des HTML-Dokuments.
- Browser zeigen nur den Inhalt zwischen dem öffnenden und schliessenden body-Tag im Browserfenster.
- Enthält weitere HTML-Elemente, welche die Information strukturieren aber **auch Scripts**, welche an der aufgeführten Stelle ausgeführt werden.
- Ein HTML-Dokument darf nur einen body-Tag haben.

Text- bzw. Informationsstrukturierung

- Mit Strukturen sind *Absätze, Titel, Überschriften, Listen* etc. gemeint.

Beispiele:

- Absatz: `<p>..</p>`
- Zeilenumbruch: `<br/>`
- Vorformatierung: `<pre>.. </pre>`
- Überschriften: `<h1>..</h1>` bis `<h6>..</h6>`
- Waagrechte Linien: `<hr/>`
- Container: `<div>...</div>` oder `<span>..</span>`

Grafiken

- Grafikformate gif, jpg, png, svg, webp... (viele weitere möglich)
- Einbinden mit:

URL

```

```



- alt-Attribut verwenden spezielle Browser z.B. für Blinde oder Suchmaschinen (z.B. Google-Bildersuche).

HTML-Links (Hyperlinks)



- Link-Element `<a>`, Attribute `href` ist Ziel, Inhalt vom Benutzer gesehen:

```
<a href="ziel">Inhalt</a>
```

Beispiel (Link zu HSLU-Seite):

```
<a href="http://www.hslu.ch">HSLU</a>
```

- Optionales `target`-Attribut legt fest, ob der Browser den Link im gleichen (`_self`) oder in einem neuen (`_blank`) Tab öffnet. Default ist gleicher Tab.

Beispiel (Link zu HSLU-Seite in neuem Tab):

```
<a href="http://www.hslu.ch" target="_blank">HSLU</a>
```

HTML-Links: Extern vs. Intern

- Externe Verknüpfung (ausserhalb des HTML-Dokuments): Angabe einer URL

Beispiel:

```
<a href="http://www.hslu.ch">HSLU</a>
```

- Interne Verknüpfungen (innerhalb des HTML-Dokuments):

Verweis mittels # und Verweis auf id-Attribut:

Beispiel:

```
<a href="#Kapitel1">Kapitel 1</a>  
...  
<p id="Kapitel1">Kapitel 1</p>
```

URLs in HTML-Dokumenten

URLs in HTML-Dokumenten sind entweder:

- **Absolut** (vollständige URLs) zur Referenzierung von Ressourcen in **anderer Domain** als die des aktuell angezeigten HTML-Dokuments.
- **Relativ** zur URL des aktuell angezeigten Dokuments (unvollständige URL). Zur Referenzierung von Ressourcen mit **gleicher Domain** wie aktuelles HTML-Dokument.

Faustregel: Wann immer möglich, relative Angaben verwenden.

Relative URLs

Angabe als URL ohne Protokoll:

- Relativ zur **Domain des aktuell geladenen Dokuments** (Basis), falls beginnend mit /.
Beispiel: `"/news/2024/artikel123.html"`
- Relativ zum **Verzeichnis des aktuell geladenen Dokuments** (Basis), falls nicht beginnend mit /.
Beispiel: `"2024/artikel123.html"`

Funktionsweise:

- Effektive Ressource bestimmt durch **Zusammensetzen von Basis und relativer URL**.
- URLs werden **clientseitig** zusammengesetzt, der Webclient hat alle Informationen.

Beispiele relativer URLs

Relativ zur aktuellen Domain: `"/news/2024/artikel123.html"`

- Aktuelles HTML-Dokument: `http://www.example.com/news/overview.html`

Basis ist Protokoll und Domain

⇒ **Effektive URL:** `http://www.example.com/news/2024/artikel123.html`

Relativ zum aktuellen HTML-Dokument: `"2024/artikel123.html"`

- Aktuelles HTML-Dokument: `http://www.example.com/news/overview.html`

Basis ist Verzeichnis (alles ausser letztes Pfadelement)

⇒ **Effektive URL:** `http://www.example.com/news/2024/artikel123.html`

Metatags



- Werden nicht zur Gestaltung, sondern zur Beschreibung des Inhalts verwendet (daher der Name: Metainformation).
- Werden im head-Bereich eingefügt.
- Prinzipieller Aufbau: `<meta name="Schlüsselwort" content="Inhalt">`

Beispiele von Metatags:

keywords	→ Schlüsselwörter (i.d.R. nicht von Suchmaschinen verwendet).
description	→ Beschreibung des Dokuments.
robots	→ Kontrolle der Suchmaschinen-Indexierung.
viewport	→ Benötigt für responsives Verhalten.

Weitere Metatags: <https://html.spec.whatwg.org/multipage/semantics.html#the-meta-element>

Sonderzeichen (Entities)

- Zur korrekten Darstellung von Sonderzeichen muss das charset Metatag korrekt gesetzt sein.

Beispiele:

- `<meta charset="utf8">` falls der UTF8-Zeichensatz verwendet wird.
`<meta charset="iso-8859-1">` falls der ISO-8859-1-Zeichensatz verwendet wird.

- Veraltete Alternative ist Zeichen speziell zu kodieren (Escape-Sequenzen):

Beispiel: aus ü wird `ü`;

- Escape-Sequenzen sind notwendig bei Zeichen, welche mit dem HTML-Markup kollidieren:
& zu `&`, < zu `<`, > zu `>`, " zu `"`, ' zu `'`.

- Liste mit Beispielen: <http://www.htmlhelp.com/reference/html40/entities/latin1.html>

Masseinheiten

- Bestimmung der Dimensionen verschiedener Elemente wie Bilder, Schriften, Ränder, Abstände etc.
- Heutzutage werden eher relative Masseinheiten verwendet.

Relative Einheiten:

- px: (Pixel) Abhängig von der **Pixeldichte** des Ausgabegeräts.
- em: relativ zur Schriftgrösse des Elements M.
- %: (Prozent) typischerweise relativ zum Elternelement (mit Ausnahmen)

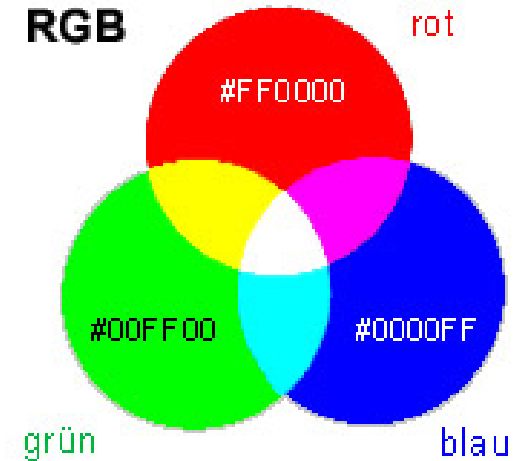
Absolute Einheiten (eher selten verwendet):

- mm: Millimeter
- in: Inch, 1 Inch entspricht 2.54 Zentimetern
- pt: Punkt, 1 Punkt entspricht 1/72 Inches

Farben

- Farben werden aus den RGB (rot-grün-blau) Wertangaben gebildet.

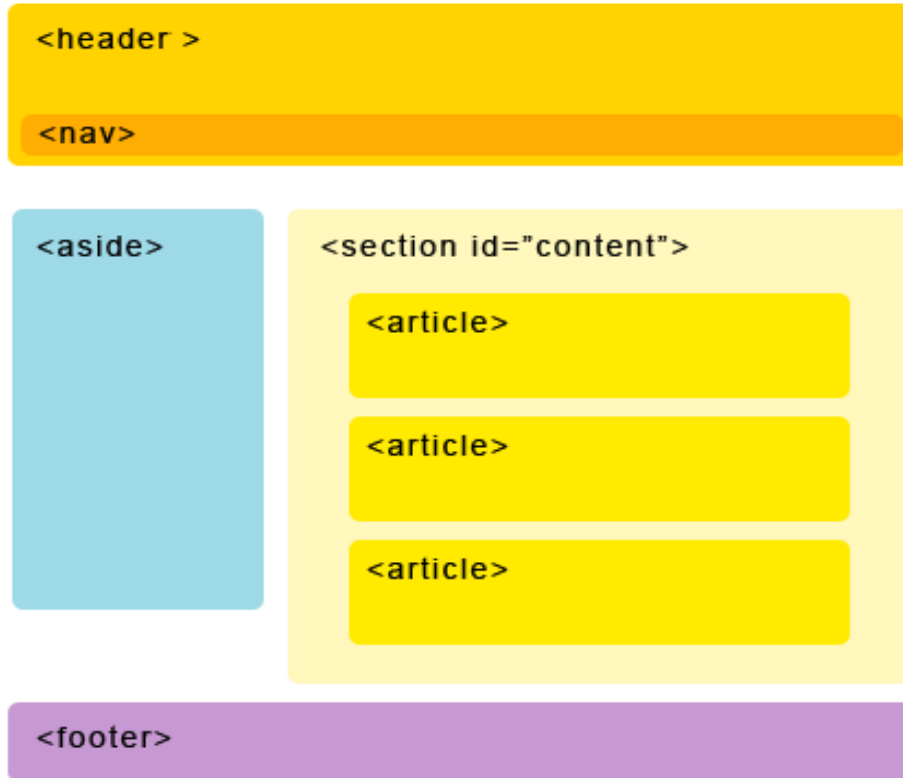
Beispiel: #00FF00 ist grün.



- Werte werden in hexadezimaler Form angegeben
- Pro Farbe sind Werte von #00 bis #FF möglich (nicht case-sensitive).
- Einige Farben sind per Namen vordefiniert (Auswahl):

Black	=	#000000	Green	=	#008000	Maroon	=	#800000	Navy	=	#000080
Silver	=	#C0C0C0	Lime	=	#00FF00	Red	=	#FF0000	Blue	=	#0000FF
Gray	=	#808080	Olive	=	#808000	Purple	=	#800080	Teal	=	#008080
White	=	#FFFFFF	Yellow	=	#FFFF00	Fuchsia	=	#FF00FF	Aqua	=	#00FFFF

Strukturierung mittels semantischer Container



Semantische Container: Elemente (1)

<main>-Element:

- Inhalt, welcher dem zentralen Thema eines Dokuments oder der zentralen Funktionalität einer Applikation entspricht.
- Darf exakt einmal in einem HTML-Dokument vorkommen.

Semantische Container: Elemente (2)

<header>-Element:

- Enthält sichtbaren Kopfbereich eines HTML-Dokuments.
- Gruppierung einleitender Inhalte (Firmenlogos, ein Motto, Links zum Impressum oder zur Kontaktseite).

<footer>-Element:

- enthält Informationen, die in HTML-Dokumenten am Ende stehen: Autor, Hinweise zum Urheberrecht, ein Link zum Impressum.
- Position ist nicht notwendigerweise am unteren Rand
→ bei Blog-Einträgen steht der footer oft neben dem Text.

Semantische Container: Elemente (3)

<article>-Element:

- Stellt in sich geschlossene Abschnitte eines Dokuments dar
 - vergleichbar mit einem Zeitungsartikel.
 - innerhalb von article-Elementen weitere strukturierende Elemente wie header, section oder footer.

<section>-Element:

- Enthält eine thematische Gruppierung von Inhalten typischerweise mit einer Überschrift.
- Dient dazu, den Inhalt oder auch einen article in semantische Abschnitte zu gliedern.

Semantische Container: Elemente (4)

<nav>-Element:

- Umschliesst insbesondere Navigationsleisten.
- Kann neben einer ungeordneten Liste mit den Verweisen auch eine Überschrift oder ähnliches enthalten.

<aside>-Element:

- Umschliesst Abschnitte einer Seite, deren Inhalt nur in einem indirekten Zusammenhang mit dem umgebenden Inhalt stehen.
- Beispiele: Randbemerkungen, Fussnoten oder Links zu weitergehenden HTML-Dokumenten.

Beispiele semantischer Container

```
<header>  
  <h1>HTML5 Seminar</h1>  
</header>
```

```
<footer>  
  <p>macro</p>  
  <p>CC 3.0 Lizenz</p>  
</footer>
```

```
<figure>  
  Diagramm, Abbildung  
</figure>
```

```
<aside>  
  <p>Weiterführende Infos</p>  
</aside>
```

```
<nav>  
  <ul>Hauptnavigation  
    <li>Startseite</li>  
  </ul>  
</nav>
```

```
<figcaption>  
  Bildunterschrift  
</figcaption>
```

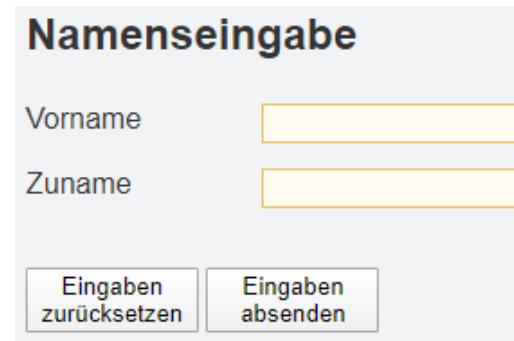
Formulare

Formulare ermöglichen es Benutzern zwischen Optionen zu wählen oder Daten wie Adressen, Termine und Texte einzugeben und diese zur Weiterverarbeitung abzusenden.

Formularelemente:

- werden mit dem `<form>`-Tag erzeugt.
- bestehen aus Formularelementen (z.B. Eingabefelder, Buttons, etc.).

Beispiel:



The example shows a form titled "Namenseingabe" with two input fields: "Vorname" and "Zuname". Below the fields are two buttons: "Eingaben zurücksetzen" and "Eingaben absenden".

(HTML-Code nächste Seite)

Formulare: Labels, Inputs und Button

```
<form>
  <h2>Namenseingabe</h2>
  <label for="vorname">Vorname</label>

  <input type="text" id="vorname">

  <label for="nachname">Nachname</label>
  <input type="text" id="nachname">

  <button>Eingaben absenden</button>
</form>
```

Attribut "for": verknüpft <label>-Element mit <input>-Feld

Attribut "id" wird für Verknüpfung mit dem Label benötigt

Button: Z.B. um Formular abzusenden (i.d.R. mit Hilft von JavaScript).

Formularelemente: Radiobutton

```
<form>
  <h2>Namenseingabe</h2>
  <label for="male" >Mann</label>

  <input type="radio" name="gender" id="male" >

  <label for="female">Frau</label>
  <input type="radio" name="gender" id="female">

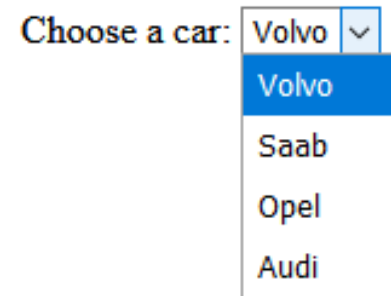
  <button>Eingaben absenden</button>
</form>
```

Attribut "name": wird u.A. verwendet
um festzulegen welche Radiobuttons
zur selben Gruppe gehören

Formularelemente: Dropdownlisten und Platzhalter

- Dropdown-Listen (<select>): Verwendung analog zu Inputelementen (insbesondere das name-Attribut):

```
<label for="cars">Choose a car:</label>
<select id="cars">
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
  <option value="mercedes">Mercedes</option>
  <option value="audi">Audi</option>
</select>
```



Quelle:

https://www.w3schools.com/tags/tag_select.asp

- Placeholder: Platzhalter für Eingabevorschläge (kein Ersatz für das Label!):

```
<input type="text" id="i" placeholder="beautiful">
```

Formularelemente: Validierung mittels HTML

Clientseitige Eingabevalidierungen können direkt mittels HTML-Attributen spezifiziert werden.

Beispiele:

- required-Attribut: Input **muss** angegeben werden:

```
<input type="text" id="name" required>
```

- type="email": Erwartet eine korrekte **Emailadresse** (nur Formatprüfung):

```
<input type="email" id="mail">
```

- min und max: Erwartet eine **Nummer X im Bereich** ($\text{min} \leq X \leq \text{max}$)

```
<input type="number" id="e" min="5" max="20">
```


Videoelemente



- Einbindung von Videos in HTML-Dokumente:

```
<video src="movie.mp4" controls width="360" height="240"  
poster="startbild.jpg"> </video>
```

- Mehrere Formate möglich: Browser zeigt das erste Video, dessen Format er unterstützt:

```
<video controls width="360" height="240" poster="startbild.jpg">  
  <source src="movie.ogv" type="video/ogg">  
  <source src="movie.mp4" type="video/mp4">  
</video>
```

- Boolesche-Attribute (HTML-Attribute ohne Wert)
 - autoplay: Wiedergabe startet automatisch (sparsam einsetzen!).
 - controls: Videosteuerung wird eingeblendet (Start/Stop/Pause/etc.).

Zusammenfassung

- HTML: Sprache zur Strukturierung und Auszeichnung von Informationen eines Dokuments.
- HTML-Standard spezifiziert Tags und Attribute zur Auszeichnung dieser Informationen.
- Diese Standardisierung ermöglicht einem Programm (z.B. ein Webbrowser) die Inhalte eines HTML-Dokuments auszuwerten und zu verarbeiten (z.B. darstellen).
- Eines der wichtigsten HTML-Elemente ist der Hyperlink, also der Verweis zu weiterführenden Informationen.
- Formulare bieten eine erste Form der Interaktivität: Sie erlauben einem Benutzer eines HTML-Dokuments, Informationen an einen Webserver zu schicken.